

SIMATS ENGINEERING

ASSESSMENT TOOL 2 (APPLICATION BASED PROBLEM SOLVING)

COURSE CODE:CSA0921

COURSE NAME: Programming in JAVA

COURSE OUTCOME COVERED

CO5. *Construct database-driven web applications by implementing JDBC connectivity to perform SQL operations such as insert, update, and delete using different statement types. (BL3, BL6)*

Assessment Tool Weightage: 20%

QUESTIONS: 1

Total Marks:50

Annexure A

APPLICATION BASED PROBLEM SOLVING

Code of Conduct:

I P.K.SALMIKSHA (192511318) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

SALMIKSHA P.K.

Annexure A

APPLICATION BASED PROBLEM SOLVING

1: Smart Library Book Registration System (Statement + PreparedStatement)

A college library is digitizing its book inventory. The librarian needs a Java application that allows new books to be added while preventing duplicate ISBN numbers from being stored.

Database Table: Books

BookID	Title	Author	ISBN	AvailableCopies
--------	-------	--------	------	-----------------

Tasks:

1. Establish JDBC connectivity with MySQL.
2. Insert five book records using PreparedStatement.
3. Before insertion, verify whether the ISBN already exists using a Statement.
4. Display an appropriate success or duplicate message.

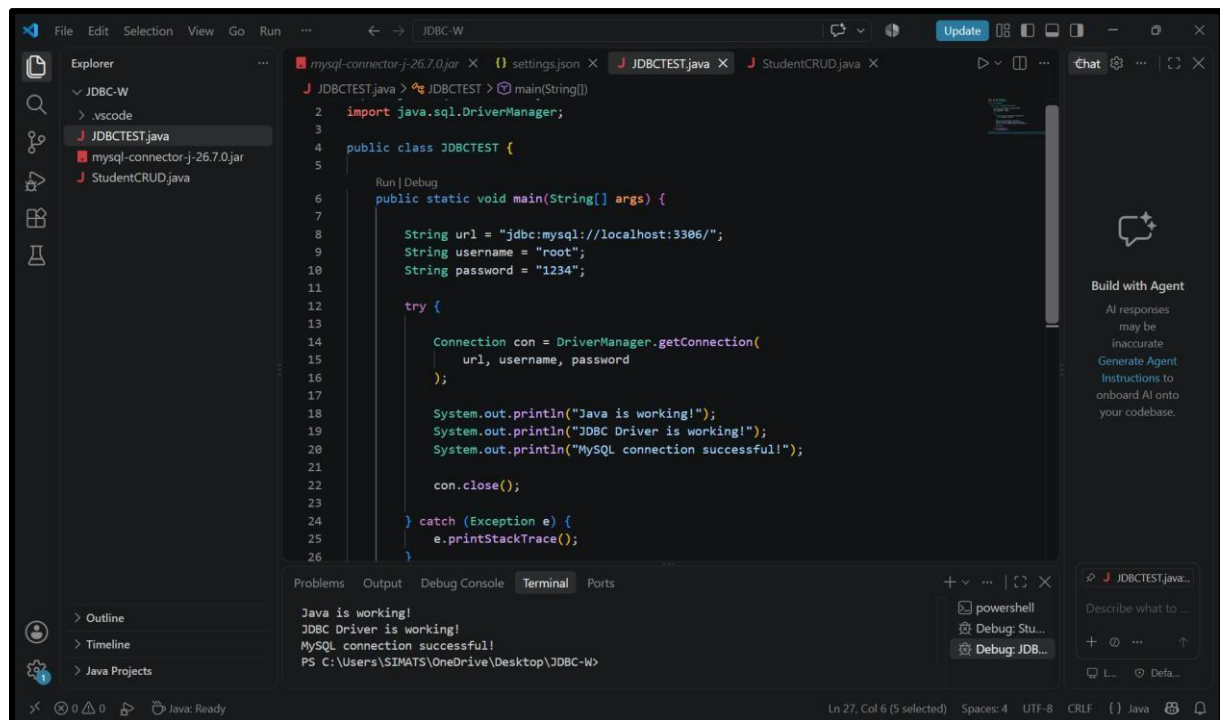
Problem-solving focus: Choosing between Statement and PreparedStatement for validation and insertion.

```
import java.sql.Connection; import
java.sql.DriverManager; public class
JDBCTEST {

    public static void main(String[] args) {
        String url = "jdbc:mysql://localhost:3306/";
        String username = "root";
        String password = "1234";
        try
        {
            Connection con = DriverManager.getConnection(
url, username, password
            );
            System.out.println("Java is working!");
            System.out.println("JDBC Driver is working!");
            System.out.println("MySQL connection successful!");    con.close();

        } catch (Exception e) {
            e.printStackTrace();
        }
    }
}
```

OUTPUT:



```
import java.sql.Connection; import
java.sql.DriverManager; import
java.sql.PreparedStatement; import
java.sql.ResultSet; import
java.sql.Statement;
```

```
public class LibraryRegistration {

    public static void main(String[] args) {

        String url = "jdbc:mysql://localhost:3306/csa09";

        String username = "root";

        String password = "1234";

        String[][] booksToInsert = {

            {"Java: The Complete Reference", "Herbert Schildt", "978-1260440232", "5"},
            {"Head First Java", "Kathy Sierra", "978-0596009205", "3"},
```

```

        {"Effective Java", "Joshua Bloch", "978-0134685991", "4"},
{"Spring in Action", "Craig Walls", "978-1617294945", "2"},
        {"Java Concurrency in Practice", "Brian Goetz", "978-0321349606", "6"}
    };

    try
    {
        Connection con = DriverManager.getConnection(url, username, password);

        System.out.println("Connected to database 'csa09' successfully.\n");

        String insertQuery = "INSERT INTO Books (Title, Author, ISBN, AvailableCopies) VALUES
        (?, ?, ?, ?)";

        PreparedStatement pstmt = con.prepareStatement(insertQuery);

        Statement stmt = con.createStatement();

        for (String[] book : booksToInsert) {
            String title = book[0];

            String author = book[1];

            String isbn = book[2];          int copies =
            Integer.parseInt(book[3]);
        }
    }
}

```

// 2. Verify whether the ISBN already exists using a Statement

```

String checkQuery = "SELECT ISBN FROM Books WHERE ISBN = " + isbn + "";
ResultSet rs = stmt.executeQuery(checkQuery);

```

// 3. Evaluate and Insert or Reject

```

if (rs.next()) {

    System.out.println(" DUPLICATE: " + title + " (ISBN: " + isbn + ") already exists.");
}

```

```

        } else {

            // 4. Insert using PreparedStatement

pstmt.setString(1, title);
pstmt.setString(2, author);
pstmt.setString(3, isbn);           pstmt.setInt(4,
copies);

            int rowsAffected = pstmt.executeUpdate();
if (rowsAffected > 0) {

                System.out.println(" SUCCESS: Inserted '" + title + "' into the library.");
            }
        }
    }

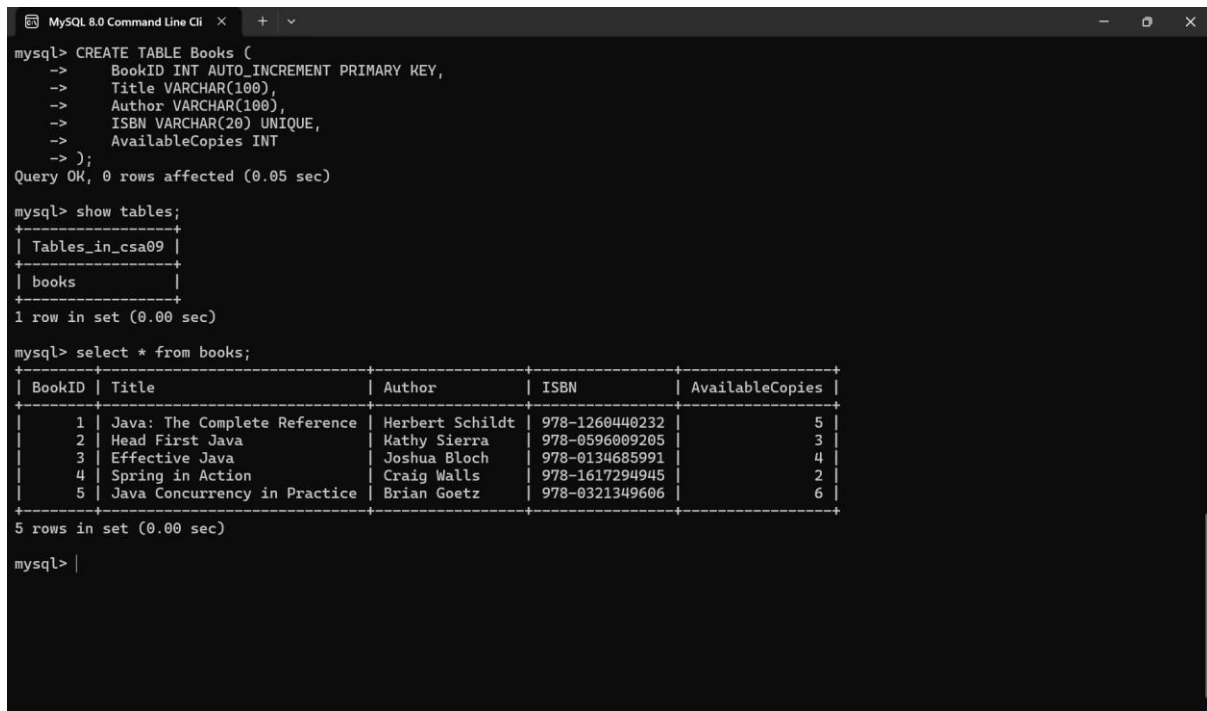
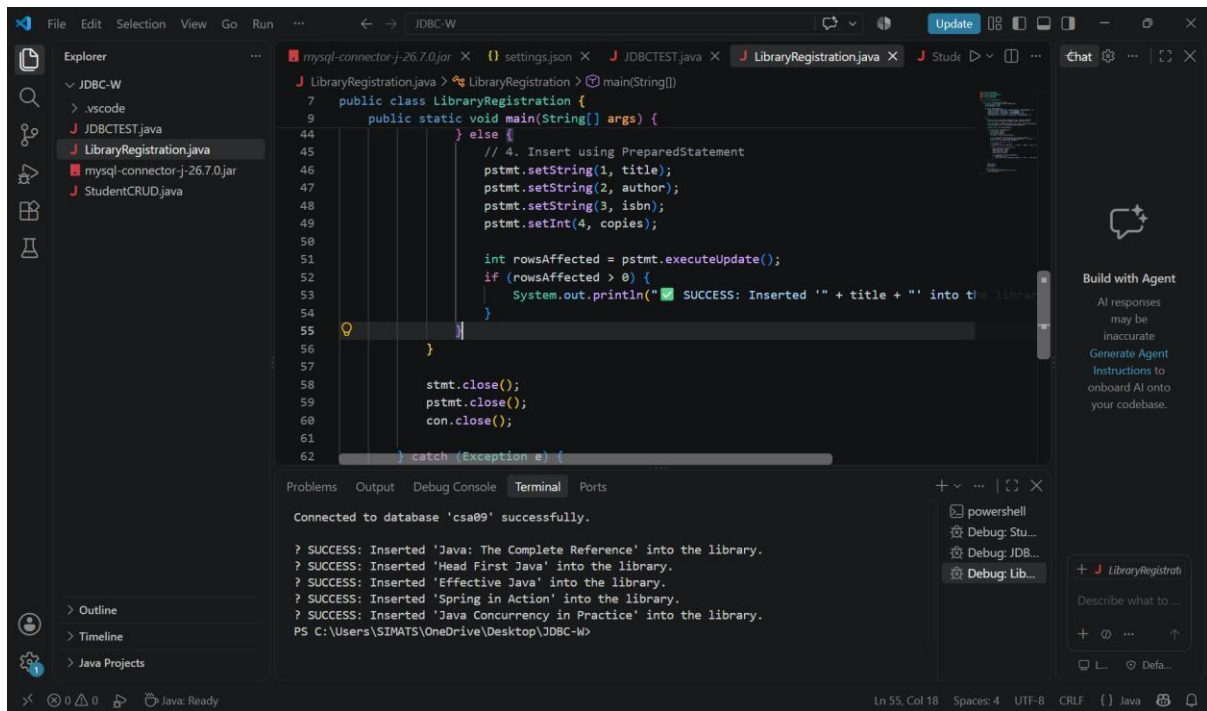
    stmt.close();
pstmt.close();           con.close();

    } catch (Exception e) {

        System.out.println("Database error occurred!");
e.printStackTrace();
    }
}
}

```

OUTPUT:



Criteria	Weightage	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Problem Analysis & Solution Design	20%	Accurately analyzes the application scenario and designs a complete JDBCbased solution with appropriate SQL operations.	Correctly analyzes the scenario with minor gaps in the solution design.	Partially understands the scenario and proposes an incomplete solution.	Unable to identify the problem requirements or design a suitable solution.
JDBC Connectivity & Statement Usage	25%	Correctly establishes database connectivity and effectively uses the appropriate JDBC statement type (Statement, PreparedStatement, or CallableStatement).	Connection and statement implementation are mostly correct with minor errors.	JDBC connectivity is partially implemented; statement usage is inconsistent.	Fails to establish database connectivity or use JDBC statements correctly.
SQL Operations & Data Integrity	20%	Implements INSERT, UPDATE, and DELETE operations accurately while maintaining data integrity and proper parameter handling.	SQL operations are correct with minor logical or syntax issues.	Performs basic SQL operations with noticeable errors in data handling.	SQL operations are incorrect or fail to modify the database as required.
Program Logic & Exception Handling	20%	Demonstrates strong logical reasoning, validates inputs, and handles exceptions effectively using appropriate techniques.	Good program logic with basic input validation and exception handling.	Limited logical flow with minimal validation or error handling.	Poor program logic with no meaningful exception handling.
Code Quality & Output Verification	15%	Produces clean, modular, well documented code and correctly verifies database results through meaningful output.	Code is organized with minor documentation or output verification gaps.	Code structure is inconsistent; output verification is incomplete.	Code is poorly structured with incorrect or missing output verification.